

From Purpose to Understanding: Connecting Theory, Data, and Engineering Decisions

My teaching philosophy begins with purpose. Students learn abstract ideas more deeply when they understand what the knowledge makes possible: what real problem it helps solve, what decision it informs, or what phenomenon it helps explain. I therefore introduce concepts through meaningful engineering and everyday applications before asking students to master the mathematical or computational machinery. When students can see why a method matters, their curiosity, imagination, active mental participation, and retention increase. This makes it easier to connect theoretical and abstract concepts to practical situations and to develop the judgment needed to use those concepts responsibly.

This philosophy grew from teaching mathematics, physics, science, and GRE preparation in Nigeria and has developed further through instructional responsibilities in Industrial Engineering at Arizona State University. My ASU experience includes Probability and Statistics for Engineering Problem Solving, Quality Control, and Advanced Quality Control. Across classroom teaching, GRE mentoring, and educational outreach, more than 300 learners have benefited from my instruction, including students who are now pursuing Ph.D. and master's degrees as well as graduates of master's programs. These experiences have convinced me that effective engineering education combines context, structured reasoning, authentic practice, timely feedback, and increasing student ownership.

1. Begin with the contribution a concept makes

Probability, statistics, optimization, and machine learning can quickly become collections of formulas unless students can see the physical or operational meaning behind them. I begin with the system and the decision before introducing the formal method: What is being observed? What is uncertain? What decision must be made? What assumptions make the model reasonable? In probability and statistics, this shifts attention from selecting a formula to understanding what a probability model represents and how variability affects an engineering conclusion. In quality engineering, it turns a control-chart statistic into a question about whether a process has meaningfully changed and what action should follow.

My experience with Advanced Quality Control reinforced the value of this approach. Student teams investigated modern statistical monitoring methods and were expected to explain the method, reproduce its logic, and connect it to an engineering context. One project examined Gaussian-mixture-model change-point detection through a semiconductor process-monitoring example in which wafer thickness transitions between operating states. The exercise required students to move among statistical assumptions, algorithmic steps, implementation, and the practical consequence of declaring a process shift. Projects of this kind make advanced methods less opaque because students can trace each mathematical object to a system behavior and an engineering decision.

2. Scaffold reasoning, then transfer ownership

Students develop independence when expert reasoning is made visible and support is removed deliberately. I organize difficult problems into a repeatable sequence: formulate the engineering question; identify knowns, unknowns, and sources of uncertainty; select a model and state its assumptions; compute or simulate; validate the result; and interpret the answer in the original context. Early assignments provide prompts for these steps. Later assignments provide only the problem statement, requiring students to decide what tools, assumptions, and diagnostic checks are necessary. In computational courses, this prevents Python or software output from becoming a substitute for engineering reasoning.

Assessment follows the same principle. I value correct results, but I also evaluate model choice, assumptions, units, diagnostic checks, interpretation, reproducibility, and communication. Low-stakes checkpoints reveal misconceptions

before high-stakes assessments. Larger projects can use staged deliverables such as problem formulation, preliminary analysis, code or model review, a final technical report, and presentation. Reproducible notebooks, version control, and simple automated checks make computational reasoning auditable while exposing students to professional engineering practice.

3. Use authentic problems to build transferable capability

Open-ended problems provide a bridge between classroom knowledge and the ambiguity students will face as engineers and researchers. I want students to encounter incomplete data, competing objectives, model mismatch, and the need to defend a recommendation. My goal is not only mastery of course content, but development of transferable skills in problem formulation, computation, scientific communication, teamwork, critical evaluation, and independent learning.

In a future undergraduate course in Manufacturing Data Analytics, teams could collect streaming vibration or temperature data from a benchtop process and build a statistical monitoring or prediction pipeline in Python. Instead of ending with a single exam, teams would produce a short conference-style paper and presentation explaining the engineering problem, model, uncertainty, limitations, and recommended action. At the graduate level, a course in Physics-Informed Machine Learning and Uncertainty-Aware Digital Twins could ask students to construct a reduced-order, neural-operator, PINN, or sparse-identification model for a physical system; calibrate it using data; quantify predictive uncertainty; validate performance; and decide when the model should be updated or rejected.

Mentoring for independence, professional development, and wellbeing

My mentoring philosophy extends the same commitment to purpose and ownership. I currently work with undergraduate, master's, recent graduate, and doctoral researchers on problems including 3D model reconstruction, active learning, generative CAD modeling, inverse modeling for microstructural characterization, and scientific machine learning. In two master's projects, I served as a co-mentor alongside my advisor, providing technical and research support that contributed to thesis development. I structure mentoring around explicit goals, regular research discussion, reproducible workflows, timely feedback, and a gradual transfer of responsibility for literature synthesis, model selection, validation, interpretation, and communication.

I also want students' research experience to contribute directly to their professional development. Training should leave students better prepared to tackle unfamiliar real-world problems, communicate across disciplines, use computational tools responsibly, and contribute independently in their chosen fields. Because mental and emotional wellbeing affect learning and research output, I pay attention to students as people as well as researchers. When appropriate, I provide support, help students identify institutional resources, protect their interests in research interactions, and create a climate in which asking for help is compatible with high expectations. Constructive feedback is central to this process: it helps students improve while also giving me information about how my own teaching and mentoring should evolve.

Inclusive access through transparent expectations and shared resources

A welcoming learning environment is one in which expectations are explicit and students have multiple legitimate ways to demonstrate competence. I use transparent rubrics, worked examples that emphasize reasoning rather than hidden tricks, early diagnostic opportunities, and normalizing office hours and project check-ins as part of the learning process. In teams, rotating technical and communication roles distribute responsibility and reduce the likelihood that one student becomes the permanent programmer, analyst, or presenter. Written, verbal, visual, and computational modes of explanation provide multiple pathways for engagement while preserving common technical standards.

My educational outreach reflects the same commitment to access. From October 2020 through July 2021, I volunteered as a GRE instructor, teaching quantitative reasoning, verbal reasoning, analytical writing, test-taking strategy, and structured problem solving. In 2021, EducationUSA Abuja invited me to deliver a session on GRE and

TOEFL/Duolingo test-taking strategies, and I returned as a resource person in 2024. I routinely share study materials, preparation resources, practical tips, and graduate-school application strategies so that students can make informed decisions and prepare more efficiently.

Future curriculum contributions and continuous improvement

I am prepared to teach undergraduate and graduate courses in probability and statistics, quality and reliability engineering, data analytics, engineering simulation, machine learning, manufacturing systems, uncertainty quantification, and systems modeling. My research also creates natural opportunities for specialized instruction in scientific machine learning, inverse problems, digital twins, physics discovery, and data-driven engineering. Regardless of course level, I want students to leave able to explain not only how an answer was obtained, but why it should be trusted, what assumptions support it, and what real decision or contribution it enables.

I treat teaching as an iterative engineering process. Course effectiveness should be evaluated with evidence such as concept-level diagnostics, performance on common rubric elements, quality of technical explanations, project completion, and structured student feedback. I use these signals to identify where students are memorizing rather than reasoning and to revise examples, scaffolds, and assessments accordingly. My goal is a learning environment in which students become increasingly curious, capable, independent, and prepared to connect rigorous theory with meaningful engineering contributions.